

Task 08 — Theory Answers

Object-Oriented Programming in C#

Part 01 — Conceptual Questions

1. IVehicle Interface — Car and Bike

Q: Why is it better to code against an interface rather than a concrete class?

A: Coding against an interface means your code depends on a contract (the interface) rather than a specific implementation. This provides flexibility because you can swap implementations without changing the calling code. It also enables polymorphism, easier unit testing (using mock objects), and reduces coupling between components. For example, with IVehicle, the same code can drive a Car, Bike, or any future vehicle without modification.

2. Abstract Class Shape — Rectangle and Circle

Q: When should you prefer an abstract class over an interface?

A: Prefer an abstract class when:

- You want to share common code (fields, non-abstract methods, constructors) among related classes.
- The classes form a clear 'is-a' hierarchy (e.g., Rectangle IS-A Shape).
- You need to define state (instance variables) that subclasses will share.
- You want to provide partial implementation so subclasses only override what differs.

A: Use an interface when you need multiple inheritance of behavior, or when unrelated classes should share a common contract.

3. IComparable — Product Sorting

Q: How does implementing IComparable improve flexibility in sorting?

A: IComparable defines a natural ordering for objects. When a class implements IComparable, standard library methods like `Array.Sort()` and `List.Sort()` can sort collections of that type without extra code. This means sorting logic is encapsulated inside the class, not scattered across the application. It also allows the type to be used in sorted collections like `SortedSet` automatically.

4. Student — Copy Constructor

Q: What is the primary purpose of a copy constructor in C#?

A: A copy constructor creates a new object that is an independent, deep copy of an existing object. Its primary purpose is to prevent unintended sharing of references (shallow copy problem). Without it, assigning one object to another copies only the reference, so changes to one affect the other. A copy constructor ensures all reference-type fields are duplicated so the two objects are fully independent.

5. Explicit Interface Implementation — Robot

Q: How does explicit interface implementation help in resolving naming conflicts?

A: When a class implements multiple interfaces that define methods with the same name, explicit interface implementation allows each method to have its own behavior depending on which interface reference is used. The method is only accessible when the object is cast to that specific interface type, preventing ambiguity. This keeps the class's public API clean while still satisfying all interface contracts.

6. Struct Account — Encapsulation

Q: What is the key difference between encapsulation in structs and classes?

A: Both structs and classes support encapsulation via private fields and public properties. However, the key differences are:

- Structs are value types: they are copied on assignment, so each variable holds its own data.
- Classes are reference types: multiple variables can point to the same object on the heap.
- Structs cannot have parameterless constructors in older C# versions and cannot inherit from other structs.
- For small, immutable data (like coordinates or a bank account snapshot), structs are more efficient.

Q: What is abstraction as a guideline? What is its relation with encapsulation?

A: As a guideline, abstraction means exposing only the essential behavior of an object while hiding internal complexity. It answers the question: 'What does this object do?' rather than 'How does it do it?'. Encapsulation is the mechanism that implements abstraction — it hides internal details behind access modifiers (private, protected) and exposes a controlled interface (public methods/properties). In short: abstraction is the principle; encapsulation is the technique that enforces it.

7. ILogger — Default Interface Implementation

Q: How do default interface implementations affect backward compatibility in C#?

A: Default interface methods (introduced in C# 8.0) allow you to add new methods to an interface without breaking existing implementations. Before C# 8, adding a method to an interface forced all implementing classes to update. With default implementations, existing classes automatically inherit the default behavior, maintaining backward compatibility. However, a class can still override the method to provide its own logic, as `ConsoleLogger` does.

8. Book — Constructor Overloading

Q: How does constructor overloading improve class usability?

A: Constructor overloading allows a class to be instantiated in multiple ways depending on available information. This improves usability by providing sensible defaults, reducing the burden on the caller to always supply all data. It also makes the API more intuitive — a caller can create a `Book` with just a

title, or with both title and author, without needing separate factory methods or nullable parameters.

Part 02 — Written Questions

1. LinkedIn Article: Abstract Classes in C#

Understanding Abstract Classes in C# — A Developer's Perspective

A: When I started learning object-oriented programming, one concept that initially seemed abstract (pun intended) but turned out to be incredibly powerful was the abstract class. Let me break it down in simple terms.

A: An abstract class in C# is a class that cannot be instantiated on its own — it exists purely to be a blueprint for derived classes. Think of it as a contract with a partial implementation. You define the 'what' in the abstract class and delegate the 'how' to the subclasses.

A: When should you use an abstract class?

- When you have shared code that multiple related classes need — put it in the abstract class.
- When you want to enforce that subclasses implement certain methods (via abstract methods).
- When your types share a true 'is-a' relationship (Dog IS-A Animal).

A: Unlike interfaces, abstract classes can contain constructors, fields, and concrete methods — giving them more power when you need to share state or behavior. The key rule: if a class has even one abstract method, the class itself must be declared abstract.

A: In my experience, abstract classes shine in scenarios like defining a Shape hierarchy where all shapes share a Display() method, but each must provide its own GetArea() implementation. This is the Template Method pattern in action.

A: Bottom line: reach for an abstract class when you want to share code among closely related classes. Reach for an interface when you want to define a capability that unrelated classes can share. Often, the best design uses both.

2. Coding Against an Interface vs. Coding Against Abstraction

What does 'coding against an interface rather than a class' mean?

A: It means writing code that depends on an interface type rather than a concrete class. For example, instead of:

```
Car myCar = new Car();
```

A: You write:

```
IVehicle myVehicle = new Car();
```

A: This means the rest of the code only knows about IVehicle — it doesn't care if it's a Car or a Bike. Swapping Car for Bike requires zero changes to the calling code. This is the Open/Closed Principle: open for extension, closed for modification.

What does 'code against abstraction, not concreteness' mean?

A: This is a broader principle from the Dependency Inversion Principle (DIP). High-level modules should not depend on low-level modules — both should depend on abstractions (interfaces or abstract classes). Concreteness means tying your code to a specific implementation. Abstraction means depending on a contract. This makes systems more flexible, testable, and maintainable because you can change implementations without touching the code that uses them.

3. Abstraction as a Guideline and Its Implementation

A: Abstraction as a guideline means: show only what is necessary, hide everything else. It is one of the four pillars of OOP (alongside Encapsulation, Inheritance, and Polymorphism).

A: How we implement abstraction through what we have studied:

- Interfaces (IVehicle, IShapeSeries): Define pure contracts — only the 'what', no implementation details exposed.
- Abstract Classes (Shape, GeometricShape): Partially abstract — share common logic but force subclasses to implement core behavior.
- Encapsulation (Account struct, Product class): Hide internal fields behind private access; expose only what callers need.
- Default Interface Methods (ILogger): Add behavior to contracts without exposing implementation details to callers.
- Factory Pattern (ShapeFactory): Hide creation logic — the caller asks for a shape by name and gets the right one without knowing the class.

A: In every pattern we studied, abstraction is the guiding principle: the caller should know as little as possible about the internals of what it uses. This reduces coupling, increases reusability, and makes systems easier to evolve.

Part 03 — Bonus

1. Self-Study Report: Key OOP Concepts Explored

A: Through this task, several important OOP concepts were explored in depth:

- Interfaces vs Abstract Classes: Interfaces define pure contracts with no state; abstract classes provide partial implementations and can hold shared fields. The choice between them depends on whether you need shared implementation or just a shared contract.
- IComparable and Sorting: Implementing IComparable gives a type a natural ordering, enabling it to work seamlessly with `Array.Sort()` and sorted collections. Custom sorting logic is encapsulated within the class.
- Copy Constructors and Deep Copy: Reference types require explicit deep copying to avoid shared mutable state. Copy constructors provide a standard mechanism for this in C#.
- Explicit Interface Implementation: Solves naming conflicts when a class implements multiple interfaces with identical method signatures. The explicit version is only callable through the interface reference.

- Default Interface Methods (C# 8+): Allow backward-compatible API evolution. Existing classes inherit the default, new classes can override it. This bridges the gap between interfaces and abstract classes.
- Factory Pattern: Decouples object creation from usage. The caller specifies what it wants; the factory decides which concrete class to instantiate. This is a direct application of coding against abstraction.
- Selection Sort: A simple $O(n^2)$ comparison-based sorting algorithm. In each pass, the minimum element from the unsorted portion is placed in its correct position. While not efficient for large datasets, it is easy to implement and understand.

2. Operator Overloading in C#

A: Operator overloading allows a class or struct to define custom behavior for standard operators such as `+`, `-`, `*`, `==`, `<`, `>`, and others. It makes user-defined types behave like built-in types syntactically.

Syntax:

```
public static ReturnType operator symbol(Type operand1, Type operand2) {
    ... }

```

A: Example — Overloading + for a Vector class:

```
public static Vector operator +(Vector a, Vector b) => new Vector(a.X +
    b.X, a.Y + b.Y);

```

A: Rules and restrictions:

- The operator keyword must be used with the static modifier.
- At least one parameter must be of the declaring class/struct type.
- Some operators must be overloaded in pairs: `==` and `!=`, `<` and `>`, `<=` and `>=`.
- The assignment operators (`+=`, `-=`) cannot be overloaded directly — they derive from `+` and `-`.
- Logical operators (`&&` and `||`) cannot be directly overloaded.

A: When to use it: Operator overloading is most appropriate for types that represent mathematical or logical concepts — vectors, matrices, complex numbers, fractions, or sets. It should be used sparingly and only when the operator's meaning is clear and intuitive for the type. Overusing it for non-mathematical types makes code harder to read and understand.